CampusDesk — Complete Solution

> The whole LCEL pipeline, every file — nothing to copy from elsewhere. Use it to **check your work**, not to skip the thinking. Build from the story first; read the hints if you get stuck.

The final layout

```
<repo folder>/
├── data/
│   ├── students.json
│   └── time_lookups.py
├── src/
│   ├── chains/
│   │   ├── __init__.py
│   │   ├── classifier.py
│   │   ├── fallback.py
│   │   ├── lookups.py
│   │   └── replies.py
│   ├── models/
│   │   ├── __init__.py
│   │   └── llm.py
│   ├── prompts/
│   │   ├── classifier.txt
│   │   └── reply.txt
│   ├── services/
│   │   ├── __init__.py
│   │   └── pipeline.py
│   ├── utils/
│   │   ├── __init__.py
│   │   ├── logger.py
│   │   └── prompts.py
│   ├── __init__.py
│   ├── config.py
│   ├── exceptions.py
│   └── schemas.py
├── tests/
│   ├── integration/
│   │   └── test_live_pipeline.py
│   └── unit/
│       ├── test_config.py
```

```
│       ├── test_fallback.py
│       ├── test_lookups.py
│       ├── test_review.py
│       └── test_schemas.py
├── .env.example
├── .gitignore
├── LICENSE
├── main.py
└── requirements.txt
```

Setup files

`requirements.txt`

```
langchain-core>=0.3
langchain-openai>=0.2
pydantic>=2.0
python-dotenv>=1.0
```

`.env.example`

```
OPENAI_API_KEY=sk-your-key-here
LOG_LEVEL=INFO
```

`.gitignore`

```
.env
.venv/
__pycache__/
*.pyc
```

Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, prompts

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str

def require_env(key):
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config():
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name):
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/utils/prompts.py
```

```
def load_prompt(name):
    with open("src/prompts" + "/" + name + ".txt") as prompt_file:
        return prompt_file.read().strip()
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

def get_model(config):
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The two fixed shapes

```
# src/schemas.py
# TicketCategory: the classifier's fixed answer. TicketRecord: one finished
# message, flags included. The flags have defaults, so a record builds even
# before the later stages fill them.
from typing import Literal

from pydantic import BaseModel, Field

class TicketCategory(BaseModel):
    category: Literal["enrollment", "financial_aid", "it_support"]
    reason: str
    sure: bool = True

class TicketRecord(BaseModel):
    student: str
    message: str
    category: str
    reply: str
    used_fallback: bool = False
    needs_human_review: bool = False
    review_reasons: list = Field(default_factory=list)
```

The fallbacks (one place)

```
# src/chains/fallback.py
# The safe paths. with_unsure_category wraps the classifier so a failed
# structured parse returns an "unsure" category instead of crashing.
```

```
# with_canned_reply wraps a reply chain so a failed model call returns a
# holding note. CANNED_REPLY is a fixed string, so the pipeline can tell a
# fallback happened by comparing against it.
from langchain_core.runnables import RunnableLambda

from src.schemas import TicketCategory

CANNED_REPLY = (
    "Thanks for your message. Our help desk is briefly unavailable, so a "
    "staff member will follow up with you shortly."
)

def _unsure_category(_inputs):
    return TicketCategory(
        category="it_support",
        reason="could not classify confidently - needs a human",
        sure=False,
    )

def with_unsure_category(classifier_chain):
    return classifier_chain.with_fallbacks([RunnableLambda(_unsure_category)])

def _canned(_inputs):
    return CANNED_REPLY

def with_canned_reply(reply_chain):
    return reply_chain.with_fallbacks([RunnableLambda(_canned)])
```

The classifier chain

```
# src/prompts/classifier.txt
Sort the student's message into exactly one help-desk office:
- enrollment: adding or dropping classes, registration, schedules.
- financial_aid: aid, disbursements, tuition, scholarships, billing.
- it_support: portal, login, password, email, or technical problems.
Give a one-sentence reason, and whether you are sure.

Message: {message}
```

```
# src/chains/classifier.py
# build_classifier_chain: prompt | model.with_structured_output, wrapped
# with the unsure fallback. classify_ticket invokes it.
from langchain_core.prompts import ChatPromptTemplate
```

```
from src.chains.fallback import with_unsure_category
from src.schemas import TicketCategory
from src.utils.logger import get_logger
from src.utils.prompts import load_prompt

def build_classifier_chain(model):
    prompt = ChatPromptTemplate.from_template(load_prompt("classifier"))
    structured = prompt | model.with_structured_output(TicketCategory)
    return with_unsure_category(structured)

def classify_ticket(model, message):
    logger = get_logger(__name__)
    category = build_classifier_chain(model).invoke({"message": message})
    logger.info(f"classified as {category.category} (sure={category.sure})")
    return category
```

The parallel lookups

```
# src/chains/lookups.py
# build_lookup_step: a RunnableParallel of three lookups that run AT THE
# SAME TIME and merge into one context dict. Each reads the local students
# file; an unknown student returns a safe "unknown" string.
import json

from langchain_core.runnables import RunnableLambda, RunnableParallel

def _load_students():
    with open("data/students.json") as students_file:
        return json.loads(students_file.read())

def _field(field_name):
    def lookup(payload):
        students = _load_students()
        record = students.get(payload["student"])
        if record is None:
            return "unknown"
        return record.get(field_name, "unknown")
    return lookup

def build_lookup_step():
    return RunnableParallel(
        status=RunnableLambda(_field("status")),
        open_requests=RunnableLambda(_field("open_requests")),
```

```
        deadline=RunnableLambda(_field("deadline")),
    )
```

The reply chains

```
# src/prompts/reply.txt
You are a university help-desk assistant for the {category} office. Reply
briefly and helpfully to the student's message, using the context. Do not
invent policies; if unsure, say a staff member will follow up.

Message: {message}
Context: {context}
```

```
# src/chains/replies.py
# build_reply_chain: prompt | model | parser, wrapped with the canned-reply
# fallback. The category is passed in so one template serves all offices.
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate

from src.chains.fallback import with_canned_reply
from src.utils.prompts import load_prompt

def build_reply_chain(model):
    prompt = ChatPromptTemplate.from_template(load_prompt("reply"))
    chain = prompt | model | StrOutputParser()
    return with_canned_reply(chain)
```

The pipeline (the one door)

```
# src/services/pipeline.py
# handle_ticket ties the four stages together: classify, look up in
# parallel, stream the reply (with fallback), and build one TicketRecord
# with the review flags. main.py and the tests both call this.
from src.chains.classifier import classify_ticket
from src.chains.fallback import CANNED_REPLY
from src.chains.lookups import build_lookup_step
from src.chains.replies import build_reply_chain
from src.schemas import TicketRecord
from src.utils.logger import get_logger

def review_reasons(category, message):
```

```
    reasons = []
    if not category.sure:
        reasons.append("classifier was not sure")
    lowered = message.lower()
    for word in ["today", "locked out", "dropped", "urgent", "immediately"]:
        if word in lowered:
            reasons.append(f"urgent signal: {word}")
    return reasons

def draft_reply(model, category, message, context):
    chain = build_reply_chain(model)
    pieces = []
    for piece in chain.stream(
        {"category": category, "message": message, "context": context}
    ):
        print(piece, end="", flush=True)
        pieces.append(piece)
    print()
    reply = "".join(pieces)
    used_fallback = reply.strip() == CANNED_REPLY
    return reply, used_fallback

def handle_ticket(model, student, message):
    logger = get_logger(__name__)
    category = classify_ticket(model, message)
    context = build_lookup_step().invoke({"student": student})
    reply, used_fallback = draft_reply(
        model, category.category, message, context
    )
    reasons = review_reasons(category, message)
    logger.info(f"record: {category.category} fallback={used_fallback} "
                f"review={len(reasons) > 0}")
    return TicketRecord(
        student=student, message=message, category=category.category,
        reply=reply, used_fallback=used_fallback,
        needs_human_review=len(reasons) > 0, review_reasons=reasons,
    )
```

The entry point and the timing demo

```
# main.py
import sys

import openai
```

```
from src.config import load_config
from src.exceptions import MissingConfigError
from src.models.llm import get_model
from src.services.pipeline import handle_ticket

QUEUE = [
    ("s-1001", "How do I add a second class this term?"),
    ("s-2002", "My financial aid hasn't arrived and tuition is due today."),
    ("s-1001", "I'm locked out of the student portal."),
]

def main():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)
    model = get_model(config)
    for student, message in QUEUE:
        print(f"\n[{student}] {message}")
        print("Reply: ", end="", flush=True)
        try:
            record = handle_ticket(model, student, message)
        except openai.AuthenticationError:
            print("Login failed — check OPENAI_API_KEY in .env.")
            sys.exit(1)
        flag = "ok"
        if record.needs_human_review:
            flag = "NEEDS HUMAN"
        print(f"  -> {record.category} | fallback={record.used_fallback} "
              f"| {flag} {record.review_reasons}")

if __name__ == "__main__":
    main()
```

```
# data/time_lookups.py
# Shows the parallel win: three 0.3s lookups one-after-another vs. together.
# Run from the repo root: python -m data.time_lookups
import time

from langchain_core.runnables import RunnableLambda, RunnableParallel

def slow(_inputs):
    time.sleep(0.3)
    return "done"
```

```
def main():
    start = time.time()
    for _ in range(3):
        slow(None)
    print(f"one after another: {time.time() - start:.2f}s")

    parallel = RunnableParallel(
        a=RunnableLambda(slow), b=RunnableLambda(slow), c=RunnableLambda(slow)
    )
    start = time.time()
    parallel.invoke({})
    print(f"in parallel:       {time.time() - start:.2f}s")

if __name__ == "__main__":
    main()
```

```
# data/students.json
{
  "s-1001": {"status": "enrolled, 12 credits",
             "open_requests": "1 open: transcript request",
             "deadline": "Add/drop ends Friday"},
  "s-2002": {"status": "enrolled, 15 credits",
             "open_requests": "none",
             "deadline": "Tuition payment due today"}
}
```

The tests

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "CAMPUSDESK_TEST_SETTING"

def test_missing_setting():
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
```

```
    print(f"case 1 (missing setting) -> {result}")

def test_empty_setting():
    os.environ[TEST_SETTING] = ""
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 2 (empty setting)   -> {result}")

def test_config_loads():
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 3 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_empty_setting()
    test_config_loads()
```

```
# tests/unit/test_schemas.py
# Run from the repo root: python -m tests.unit.test_schemas
from pydantic import ValidationError

from src.schemas import TicketCategory, TicketRecord

def test_category_builds():
    category = TicketCategory(category="enrollment", reason="adds a class")
    result = "OK"
    if category.sure is not True:
        result = "WRONG"
    print(f"case 1 (good category, sure defaults True) -> {result}")

def test_category_rejects_unknown():
    result = "WRONG"
    try:
        TicketCategory(category="sports", reason="x")
    except ValidationError:
        result = "OK"
    print(f"case 2 (unknown category rejected)         -> {result}")

def test_record_builds_with_defaults():
```

```
    record = TicketRecord(student="s-1", message="m", category="it_support",
                          reply="r")
    result = "OK"
    if record.used_fallback or record.needs_human_review:
        result = "WRONG"
    if record.review_reasons != []:
        result = "WRONG"
    print(f"case 3 (record defaults are safe)          -> {result}")

if __name__ == "__main__":
    test_category_builds()
    test_category_rejects_unknown()
    test_record_builds_with_defaults()
```

```
# tests/unit/test_lookups.py
# The parallel lookups: merge + the parallel speed-up. No model.
# Run from the repo root: python -m tests.unit.test_lookups
import time

from langchain_core.runnables import RunnableLambda, RunnableParallel

from src.chains.lookups import build_lookup_step

def test_merges_three_fields():
    out = build_lookup_step().invoke({"student": "s-1001"})
    result = "OK"
    if set(out.keys()) != {"status", "open_requests", "deadline"}:
        result = "WRONG"
    if out["status"] == "unknown":
        result = "WRONG"
    print(f"case 1 (merges three lookups)   -> {result}")

def test_unknown_student_is_safe():
    out = build_lookup_step().invoke({"student": "nobody"})
    result = "OK"
    if out["status"] != "unknown":
        result = "WRONG"
    print(f"case 2 (unknown student -> safe) -> {result}")

def test_runs_in_parallel():
    def slow(_inputs):
        time.sleep(0.3)
        return "x"
    step = RunnableParallel(
        a=RunnableLambda(slow), b=RunnableLambda(slow), c=RunnableLambda(slow)
    )
```

```
    start = time.time()
    step.invoke({})
    elapsed = time.time() - start
    result = "OK"
    if elapsed > 0.6:
        result = "WRONG"
    print(f"case 3 (3x0.3s runs under 0.6s) -> {result}")

if __name__ == "__main__":
    test_merges_three_fields()
    test_unknown_student_is_safe()
    test_runs_in_parallel()
```

```
# tests/unit/test_fallback.py
# The two fallbacks fire on failure. No model. Run from the repo root:
#   python -m tests.unit.test_fallback
from langchain_core.runnables import RunnableLambda

from src.chains.fallback import (
    CANNED_REPLY,
    with_canned_reply,
    with_unsure_category,
)

def _boom(_inputs):
    raise RuntimeError("model down")

def test_canned_reply_on_failure():
    chain = with_canned_reply(RunnableLambda(_boom))
    reply = chain.invoke({"x": 1})
    result = "OK"
    if reply != CANNED_REPLY:
        result = "WRONG"
    print(f"case 1 (reply fails -> canned)      -> {result}")

def test_unsure_category_on_failure():
    chain = with_unsure_category(RunnableLambda(_boom))
    category = chain.invoke({"message": "x"})
    result = "OK"
    if category.sure is not False:
        result = "WRONG"
    print(f"case 2 (classify fails -> unsure)   -> {result}")

if __name__ == "__main__":
    test_canned_reply_on_failure()
    test_unsure_category_on_failure()
```

```
# tests/unit/test_review.py
# The review-flag logic, no model. Run from the repo root:
#   python -m tests.unit.test_review
from src.schemas import TicketCategory
from src.services.pipeline import review_reasons

def sure_category():
    return TicketCategory(category="enrollment", reason="r", sure=True)

def unsure_category():
    return TicketCategory(category="it_support", reason="r", sure=False)

def test_clean_message_no_flag():
    reasons = review_reasons(sure_category(), "How do I add a class?")
    result = "OK"
    if reasons != []:
        result = "WRONG"
    print(f"case 1 (clean message -> no flag)   -> {result}")

def test_urgent_message_flagged():
    reasons = review_reasons(sure_category(), "Tuition is due today!")
    result = "OK"
    if len(reasons) == 0:
        result = "WRONG"
    print(f"case 2 (urgent word -> flagged)     -> {result}")

def test_unsure_flagged():
    reasons = review_reasons(unsure_category(), "a normal message")
    result = "OK"
    if len(reasons) == 0:
        result = "WRONG"
    print(f"case 3 (unsure -> flagged)          -> {result}")

if __name__ == "__main__":
    test_clean_message_no_flag()
    test_urgent_message_flagged()
    test_unsure_flagged()
```

```
# tests/integration/test_live_pipeline.py
# The whole pipeline against the real model. Run from the repo root:
#   python -m tests.integration.test_live_pipeline
from src.config import load_config
from src.models.llm import get_model
from src.services.pipeline import handle_ticket

def test_routes_enrollment(model):
```

```
    record = handle_ticket(model, "s-1001", "How do I add a class?")
    result = "WRONG"
    if record.category == "enrollment":
        result = "OK"
    print(f"case 1 (enrollment routed) -> {result}")

def test_urgent_is_flagged(model):
    record = handle_ticket(
        model, "s-2002", "My aid is late and tuition is due today."
    )
    result = "WRONG"
    if record.needs_human_review:
        result = "OK"
    print(f"case 2 (urgent flagged)    -> {result}")

if __name__ == "__main__":
    config = load_config()
    model = get_model(config)
    test_routes_enrollment(model)
    test_urgent_is_flagged(model)
```

Expected unit output (all `OK`):

```
case 1 (missing setting) -> OK
case 2 (empty setting)   -> OK
case 3 (config loads)    -> OK
```

```
case 1 (good category, sure defaults True) -> OK
case 2 (unknown category rejected)         -> OK
case 3 (record defaults are safe)          -> OK
```

```
case 1 (merges three lookups)   -> OK
case 2 (unknown student -> safe) -> OK
case 3 (3x0.3s runs under 0.6s) -> OK
```

```
case 1 (reply fails -> canned)      -> OK
case 2 (classify fails -> unsure)   -> OK
```

```
case 1 (clean message -> no flag)   -> OK
case 2 (urgent word -> flagged)     -> OK
case 3 (unsure -> flagged)          -> OK
```

And `python -m data.time_lookups` prints roughly:

```
one after another: 0.90s
in parallel:       0.30s
```

That's the whole pipeline. If each message is classified into the fixed shape and routed, the lookups run in parallel, a failed model call falls back to the canned reply, and unsure or urgent messages are flagged for a human — you've built it.